

Resource Allocation System – Iteration 36

Common Available Time Suggestions

Najib Attar

May 2026

1 Iteration Goal

The goal of Iteration 36 is to add common available time suggestions for rejected `CommitteeMeetingRequest` objects.

In Iteration 35, the system added `ParticipantAvailabilityRule`. That rule checks whether all required participants of a committee meeting are available at the requested time. However, if a committee meeting was rejected because a participant or the requested room was unavailable, the system only reported the rejection.

Iteration 36 improves this behavior by suggesting possible alternative time slots. These suggestions are additional information only. They do not change the approval or rejection decision of the request.

This iteration does not implement least-change optimization, closest-time ranking, alternative room search, global scheduling optimization, or automatic rescheduling.

2 Changes from Previous Iteration

The main change from Iteration 35 is the addition of a scheduling assistance service for rejected committee meeting requests.

The following changes were introduced:

- Added `MeetingTimeSuggestion`.
- Added `MeetingTimeSuggestionService`.
- Updated `AllocationService` so rejected `CommitteeMeetingRequest` objects can receive suggestions.
- Suggestions are added to the request lifecycle history.
- `request_results.csv` exports the suggestion text through the existing request history output.

- The Flask Allocation Summary displays exported suggestion text.
- Existing rules and allocation strategies remain unchanged.

The approve/reject behavior remains the same. A committee meeting that cannot be scheduled at the requested time is still rejected, but the user can now see possible alternative time slots.

3 Scenario

A student requests a committee meeting for a thesis presentation. The meeting requires several professors or academic participants. The requested time may fail because one participant is busy or because the requested room is already allocated.

For example, request 41 asks for a committee meeting on Monday from 10:00 to 11:00. One required participant is busy on Monday from 10:00 to 12:00. The request is rejected by `ParticipantAvailabilityRule`.

After Iteration 36, the system still rejects the request, but it also suggests alternative time slots such as:

- Monday 09:00-10:00 in space 301
- Monday 12:00-13:00 in space 301
- Monday 12:30-13:30 in space 301

This supports the professor's idea of helping users find a suitable time for meetings involving multiple professors.

4 Modules and Their Tasks

Module	Task
<code>CommitteeMeetingRequest</code>	Stores the meeting topic, preferred time slot, duration, and required participant IDs.
<code>MeetingTimeSuggestion</code>	Represents one suggested alternative time slot and the related space information.
<code>MeetingTimeSuggestionService</code>	Scans working hours and finds available candidate slots for rejected committee meeting requests.
<code>UserAvailabilityService</code>	Checks whether each required participant is available based on working hours and busy slots.
<code>AllocationService</code>	Calls the suggestion service after strategy processing for rejected committee meeting requests.
<code>Allocation</code>	Provides existing room allocation data used to avoid room conflicts in suggestions.
<code>RequestResultWriter</code>	Exports request history, including suggestion text, through the existing result output.
Flask Dashboard	Displays exported suggestion text in the Allocation Summary without computing suggestions itself.

Table 1: Main modules involved in Iteration 36

5 Common Available Time Suggestion Design

The new `MeetingTimeSuggestion` model represents one suggested alternative time. It stores the suggested `TimeSlot`, the related space ID, and a reason or note.

An example suggestion is:

Monday 12:00-13:00 in space 301

The `MeetingTimeSuggestionService` is responsible for generating these suggestions. It scans possible candidate slots and checks whether each candidate is valid for both the required participants and the requested room.

For every candidate slot, the service checks:

- all required participants are available,
- the requested room is available,
- the slot is inside working hours,
- the slot does not overlap participant busy slots,
- the slot does not overlap room allocations.

The service reuses `UserAvailabilityService` for participant availability. This avoids duplicating working-hours or busy-slot logic.

6 Suggestion Generation Policy

Policy Item	Value
Working days	Monday-Friday
Working hours	09:00-17:00
Scan step	30 minutes
Meeting duration	Preserved from original request
Participant check	All required participants must be available
Room check	Requested room must be available
Maximum suggestions	3

Table 2: Suggestion generation policy

The scan preserves the original meeting duration. For a one-hour meeting, valid candidate starts include 09:00, 09:30, 10:00, and so on until the meeting still fits fully inside the working day.

The system uses the same half-open interval behavior as the rest of the scheduling logic. Therefore, 09:00-10:00 and 10:00-11:00 do not conflict, while 10:00-11:00 and 10:30-11:30 do conflict.

7 Integration with AllocationService

Suggestions are generated in `AllocationService` after the selected strategy finishes processing requests.

This integration point keeps responsibilities separated:

- Rules remain responsible for validation.
- Strategies remain responsible for allocation order and allocation behavior.
- `DataLoader` remains responsible only for loading data.
- Flask displays suggestion results but does not calculate them.
- `MeetingTimeSuggestionService` handles the suggestion logic.

For a rejected `CommitteeMeetingRequest`, `AllocationService` calls `MeetingTimeSuggestionService`. If suggestions are found, they are added to the request lifecycle history. If no suggestion is found, the history receives the message:

No available alternative time found.

An example suggestion history entry is:

Suggested alternative times: Monday 09:00-10:00 in space 301; Monday
12:00-13:00 in space 301; Monday 12:30-13:30 in space 301

8 Prototype Test Case

Request	Condition	Result Before Iteration 36	Result After Iteration 36
41	Required participant busy Monday 10:00-12:00	Rejected only	Rejected with suggested alternative times
40	All required participants free	Approved	Approved, no suggestions needed

Table 3: Prototype test cases for Iteration 36

Request 41 remains rejected because a required participant is unavailable at the requested time. After this iteration, the request also receives suggested alternative times.

Request 40 remains approved because all required participants and the requested room are available. Since the request is approved, no suggestions are needed.

9 Iteration Comparison

Feature	Before Iteration 36	After Iteration 36
Rejected committee meetings	Rejected with a reason only	Rejected with a reason and possible alternative time suggestions
Participant availability	Checked by <code>ParticipantAvailabilityRule</code>	Still checked by <code>ParticipantAvailabilityRule</code>
Room availability	Checked by <code>AvailabilityRule</code>	Still checked by <code>AvailabilityRule</code> ; suggestions also avoid room allocation conflicts
Suggestion logic	Not implemented	Implemented in <code>MeetingTimeSuggestionService</code>
Allocation strategies	Process requests normally	Remain unchanged
Web dashboard	Shows request results and rejection reasons	Also displays exported suggestion text when available
Least-change optimization	Not implemented	Still not implemented
Alternative room search	Not implemented	Still not implemented

Table 4: Comparison before and after Iteration 36

10 Updated Class Diagram

The updated class diagram includes `CommitteeMeetingRequest`, `MeetingTimeSuggestion`, `MeetingTimeSuggestionService`, `UserAvailabilityService`, `UserBusySlot`, `AllocationService`, `Allocation`, and `TimeSlot`.

It shows that `MeetingTimeSuggestionService` creates `MeetingTimeSuggestion` objects and uses `UserAvailabilityService`. It also shows that `AllocationService` uses `MeetingTimeSuggestionService` after a rejected `CommitteeMeetingRequest`.

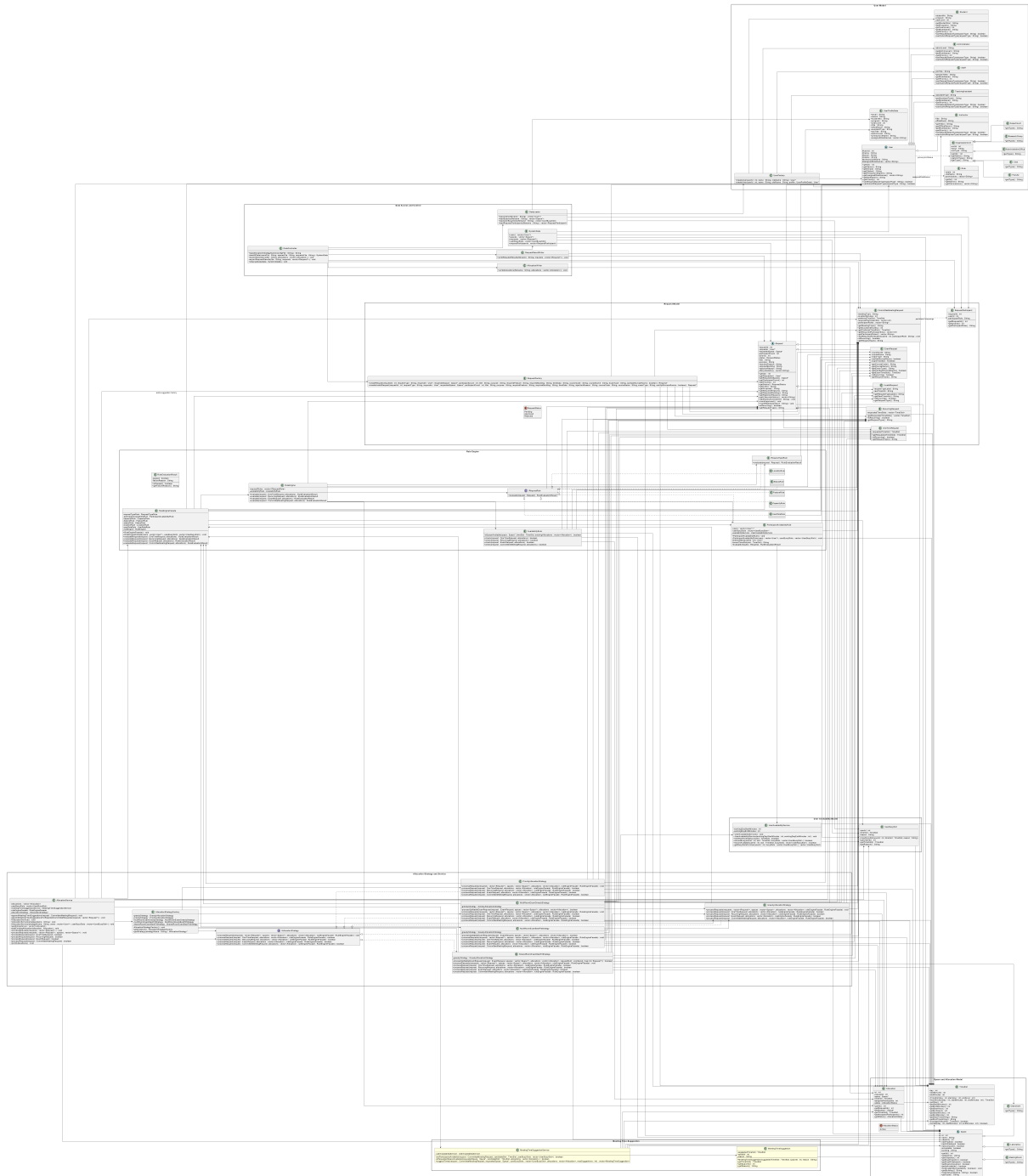
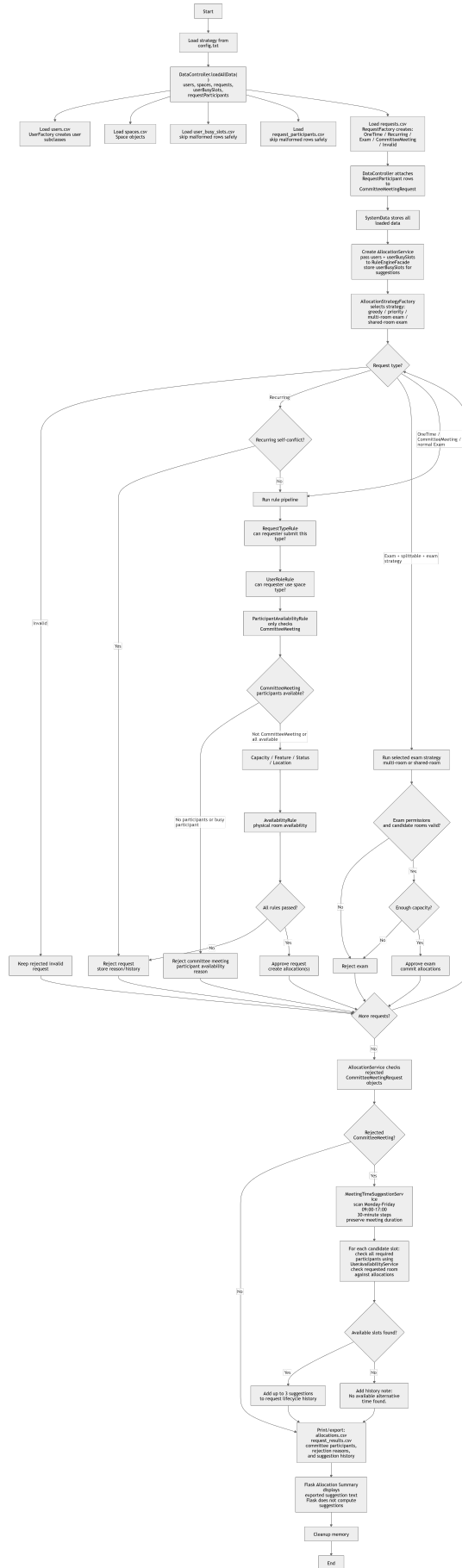


Figure 1: Updated class diagram for Iteration 36

11 Updated Workflow Diagram

The updated workflow shows that `CommitteeMeetingRequest` is processed normally through the rule pipeline. `ParticipantAvailabilityRule` may reject the request if a required participant is unavailable.

After strategy processing, `AllocationService` checks rejected committee meeting requests. `MeetingTimeSuggestionService` scans working hours, checks candidate slots against participant availability and room allocations, and adds suggestions to the request history. The suggestion text is then exported through `request_results.csv` and displayed by the web Allocation Summary.



12 Prototype Implementation

The prototype implementation confirms that common available time suggestions are now available for rejected committee meeting requests.

The implementation adds a new service instead of changing the existing rule or strategy design. `ParticipantAvailabilityRule` still performs exact-time validation. `AvailabilityRule` still checks physical room availability. Allocation strategies continue to process requests using their existing behavior.

When a committee meeting is rejected, `AllocationService` asks `MeetingTimeSuggestionService` for up to three valid alternatives. The service scans Monday through Friday from 09:00 to 17:00 in 30-minute steps and preserves the original meeting duration. It checks all required participants through `UserAvailabilityService` and checks the requested room against existing allocations.

The Flask dashboard was updated only as a supporting display feature. It shows suggestion text that was already produced by the C++ backend. Flask does not compute participant availability or meeting suggestions.

13 Current Limitation and Future Work

This iteration provides first-available suggestions only. It does not rank suggestions by closeness to the original requested time, least disruption, participant priority, or institutional preference.

The current implementation also uses the requested room only. If that room is unavailable, the service does not search for another suitable room. Alternative room search can be added in a future iteration.

Future work may include:

- closest-time ranking,
- least-change suggestion strategy,
- alternative room search,
- global scheduling optimization,
- rescheduling support,
- interactive web-based suggestion selection.

14 Summary

Iteration 36 adds common available time suggestions for rejected `CommitteeMeetingRequest` objects. The new `MeetingTimeSuggestionService` scans working hours and finds candidate slots where all required participants and the requested room are available.

This iteration keeps the design clean by adding scheduling assistance as a separate service. It does not change approval or rejection behavior, and it does not modify allocation strategies. Rejected committee meeting requests are still rejected, but now they can include helpful alternative time suggestions.

The system is now better prepared for future scheduling features such as closest-time suggestions, least-change optimization, and alternative room search.